

Mock Code Challenge - Coffee Shop (Object Relationships)

- Due No Due Date
- Points 1
- Submitting a website url

 <https://github.com/learn-co-curriculum/python-p3-mock-challenge-coffee-shop>  <https://github.com/learn-co-curriculum/python-p3-mock-challenge-coffee-shop/issues/new>

For this assignment, we'll be working with a Coffee shop-style domain.

We have three models: `Coffee` , `Customer` , and `Order` .

For our purposes, a `Coffee` has many `Order` s, a `Customer` has many `Order` s, and a `Order` belongs to a `Customer` and to a `Coffee` .

`Coffee` - `Customer` is a many to many relationship.

Note: You should draw your domain on paper or on a whiteboard *before you start coding*. Remember to identify a single source of truth for your data.

Topics

- Classes and Instances
- Class and Instance Methods
- Variable Scope
- Object Relationships
- lists and list Methods

Instructions

To get started, run `pipenv install` while inside of this directory. Then run `pipenv shell` to jump into the shell.

Build out all of the methods listed in the deliverables. The methods are listed in a suggested order, but you can feel free to tackle the ones you think are easiest. Be careful: some of the later methods rely on earlier ones.

Remember! This code challenge has tests to help you check your work. You can run `pytest` to make sure your code is functional before submitting.

We've provided you with a tool that you can use to test your code. To use it, run `python debug.py` from the command line. This will start a `ipdb` session with your classes defined. You can test out the methods that you write here. You can add code to the `debug.py` file to define variables and create sample instances of your objects.

Writing error-free code is more important than completing all of the deliverables listed - prioritize writing methods that work over writing more methods that don't work. You should test your code in the console as you write.

Similarly, messy code that works is better than clean code that doesn't. First, prioritize getting things working. Then, if there is time at the end, refactor your code to adhere to best practices. When you encounter duplicated logic, extract it into a shared helper method.

Before you submit! Save and run your code to verify that it works as you expect. If you have any methods that are not working yet, feel free to leave comments describing your progress.

Deliverables

Write the following methods in the classes in the files provided. Feel free to build out any helper methods if needed.

Initializers and Properties

Customer

- `Customer __init__(self, name)`
 - Customer is initialized with a name
- `Customer property name`
 - Returns customer's name
 - Names must be of type `str`
 - Names must be between 1 and 15 characters, inclusive
 - Should **be able** to change after the customer is instantiated

Coffee

- `Coffee __init__(self, name)`
 - Coffee is initialized with a name
- `Coffee property name`
 - Returns the coffee's name
 - Names must be of type `str`
 - Names length must be greater or equal to 3 characters
 - Should **not be able** to change after the coffee is instantiated
 - *hint: `hasattr()`*

Order

- `Order __init__(self, customer, coffee, price)`
 - Order is initialized with a `Customer` instance, a `Coffee` instance, and a price
- `Order property price`
 - Returns the price for the order
 - Prices must be of type `float`
 - Price must be a number between 1.0 and 10.0, inclusive
 - Should **not be able** to change after the order is instantiated
 - *hint: `hasattr()`*

Object Relationship Methods and Properties

Order

- `Order property customer`
 - Returns the customer object for that order
 - Must be of type `Customer`
- `Order property coffee`
 - Returns the coffee object for that order
 - Must be of type `Coffee`

Coffee

- `Coffee orders()`
 - Returns a list of all orders for that coffee
 - Orders must be of type `Order`
- `Coffee customers()`
 - Returns a **unique** list of all customers who have ordered a particular coffee.
 - Customers must be of type `Customer`

Customer

- `Customer orders()`
 - Returns a list of all orders for that customer
 - Orders must be of type `Order`
- `Customer coffees()`
 - Returns a **unique** list of all coffees a customer has ordered
 - Coffees must be of type `Coffee`

Aggregate and Association Methods

Customer

- `Customer create_order(coffee, price)`
 - Receives a **coffee object** and a **price number** as arguments
 - Creates and returns a new `Order` instance and associates it with that customer and the coffee object provided.

Coffee

- `Coffee num_orders()`
 - Returns the total number of times a coffee has been ordered
 - Returns `0` if the coffee has never been ordered
- `Coffee average_price()`
 - Returns the average price for a coffee based on its orders
 - Returns `0` if the coffee has never been ordered
 - Reminder: you can calculate the average by adding up all the orders prices and dividing by the number of orders

Bonus: Aggregate and Association Method

- `Customer` classmethod `most_aficionado(coffee)`
 - Receives a **coffee object** argument
 - Returns the `Customer` instance that has spent the most money on the coffee instance provided as argument.
 - Returns `None` if there are no customers for the coffee instance provided.
 - *hint: will need a way to remember all `Customer` objects*
 - Uncomment lines 137-147 in the `customer_test` file

Bonus: For any invalid inputs raise an `Exception` .

- First, **comment out** the following lines
 - `customer_test.py`
 - lines 25-26, 40-41, and 44-45
 - `coffee_test.py`
 - lines 34-35
 - `order_test.py`
 - lines 46-47
- Then, **uncomment** the following lines in the test files
 - `customer_test.py`
 - lines 31-32, 48-49, and 52-53
 - `coffee_test.py`
 - lines 22-23, 26-27, and 38-39
 - `order_test.py`
 - lines 32-33, 36-37, and 50-51